

Home

W1

W2

W3

W4

W5

W6

W7

## W03 Learning Activities: CSS Layout – Position

### Overview

Elements are positioned on a document using the **normal flow** by default. Sometimes, the layout calls for placing items by absolute or fixed positions. It can be tempting to use absolute positioning for everything on a page, because you can specify exactly where each item should be. But, doing this usually causes unintended outcomes, especially if the content of the page changes, or if it is viewed on different devices.

Because absolute or fixed positions can lead to unintended outcomes, you should limit using them, and when you do, use them only for specific areas and not the entire document.

The following is an example of using the **position** property to make the text "New Here?" and "Learn What We Believe" overlay on top of an image in the same HTML container:



Example of text overlaying an image in the same container

### Prepare

CSS positioning allows you to control exactly where elements appear on a webpage. Three important positioning methods are **relative positioning**, **absolute positioning**,

and **fixed positioning**. Understanding how these work together helps you create precise layouts and interactive elements.

All positioning methods use the same four properties to set location: **top**, **bottom**, **left**, and **right**. These properties tell the browser how far to move the element from each edge of its reference point.

## Relative Positioning

When you use **position: relative**, the element is moved from its normal position but still takes up its original space in the document flow. Think of it as moving a book on a table while keeping its original spot reserved. Other elements will not move into the space that the relatively positioned element originally occupied.

- **Keeps original space in document flow** – Other elements stay in their normal positions
- **Moves relative to its normal position** – Uses where the element would naturally appear as the starting point
- **Creates positioning context for children** – Child elements with absolute positioning will use this as their reference point
- **Can overlap other elements** – May cover other content when moved from its original location

The following example moves the text 10 pixels down and 20 pixels to the right from where it would normally appear, but other elements still act as if the text is in its original location.

```
.shifted-text {  
  position: relative;  
  top: 10px;  
  left: 20px;  
  background-color: yellow;  
}
```

## Absolute Positioning

When you use **position: absolute**, the element is positioned relative to its closest positioned parent element. If no parent element has positioning set, the element will be positioned relative to the entire webpage body. The element is removed from the normal document flow, which means other elements will act as if it does not exist.

- **Removes element from document flow** – Other elements fill the space as if the positioned element is not there
- **Positioned relative to closest positioned parent** – Looks for the nearest parent with position set to relative, absolute, or fixed
- **Stays with the page content when the page scrolls** – The element moves with the page content when users scroll
- **Can overlap other elements** – May cover or be covered by other page content

The following example creates a popup window that appears 50 pixels from the top and 20 pixels from the right edge of its positioned parent element.

```
.popup-window {  
  position: absolute;  
  top: 50px;  
  right: 20px;  
  width: 200px;  
  background-color: white;  
  border: 1px solid gray;  
  padding: 10px;  
}
```

## Fixed Positioning

When you use **position: fixed**, the element is positioned relative to the browser window itself. The element stays in the exact same place on the screen even when users scroll up or down the page. This makes fixed positioning perfect for navigation bars, headers, or buttons that should always remain visible.

- **Removes element from document flow** – Other elements fill the space as if the positioned element is not there
- **Positioned relative to browser window** – Always uses the viewport (visible area) as the reference point
- **Stays in same screen location when scrolling** – Does not move when users scroll the page
- **Always visible to users** – Remains on screen regardless of page length or scroll position

The following example creates a header that stays at the top of the browser window even when users scroll down the page. The **z-index** property ensures the header appears above other page content.

```
.sticky-header {  
  position: fixed;  
  top: 0;  
  left: 0;  
  width: 100%;  
  height: 60px;  
  background-color: blue;  
  color: white;  
  z-index: 100;  
}
```

## Images and position/sizing

CSS is often used to set the position and size of an image. When setting the size of an image, remember that images can become pixelated or distorted if you change the aspect ratio of their original dimensions. To avoid this, typically only set either the width or height in CSS, preserving the aspect ratio.

When an image is within a container that has defined dimensions, using **width: 100%** on the image is a standard approach to make it fill the container.

To make sure the images in your document have good aspect ratios by default, it is recommended that you start with the following CSS declaration for all of your work in this class to help control the aspect ratio of images.

```
img {  
  width: 100%;  
  height: auto;  
}
```

The CSS above sets **all** image widths in the document to 100% of the **width** of their containers and the **height** to auto, thus maintaining the image's intrinsic aspect ratio.

## Activity Instructions

### Step 1: File Setup

1. Create a file named "**overlay.html**" in the **week03** folder.
2. Add a CSS file named "**overlay.css**" in the **week03/styles** folder.

### Step 2: HTML

1. Give the **overlay.html** document a basic and valid HTML structure.
2. Be sure to **link** the CSS file in your HTML document **head**.
3. Copy this HTML into the file

```
<main>
  <h1>CSS Layout: Position Absolute</h1>
  <div class="herodiv">
    
      <h2>Brazil</h2>
      <p>Language: <span id="lan">Portuguese</span></p>
      <p>Size: <span id="size">8.5m km<sup>2</sup></span></p>
      <p>Visited: <span id="visit">No</span></p>
    </section>
  </div>
</main>
<footer>Footer Content in Fixed Position with bottom set at 0 pixels.</foo
```

### Step 3: CSS

1. Copy this CSS into the **overlay.css** file.

```
* {
  margin: 0;
  padding: 0;
}

main {
  max-width: 800px;
  margin: 2rem auto;
  font-family: Arial, Helvetica, sans-serif;
}

h1 {
  text-align: center;
  font-size: 1.5rem;
  text-shadow: 3px 3px 5px #777;
}

.herodiv {
  margin: 10px auto;
  position: relative; /* If removed, what happens? */
}

.herodiv img {
  width: 100%;
}
```

```
height: 300px;
object-fit: cover;
}

.stats {
  position: absolute;
  top: 25px;
  right: 25px;
  background: radial-gradient(#eee, #323A06);
  border: 1px solid #000;
  padding: 0.65rem;
  box-shadow: 0 0 30px #111;
  opacity: 0.9;
}

h2 {
  text-shadow: 0 0 15px #333;
  border-bottom: 1px solid #555;
  margin-bottom: 0.3rem;
}

footer {
  position: fixed;
  bottom: 0;
  margin: 1rem;
  border-top: 1px solid #000;
  padding: 0.5rem;
  color: navy;
  font-weight: 700;
  font-style: italic;
}
```

2. Use Live/Five Server to open your page in your browser for viewing.
3. Using CSS, move the **section class="stats"** display box from the upper right to display on the lower left of the image.

► Check Your Understanding

4. Remove the **position: relative;** property from the container **div** (this div has a class of **herodiv** assigned to it).  
What happens?

▼ Check Your Understanding

The information **section** overlay moves to the bottom left of the document instead of the bottom left of the **div.herodiv** container. This happens because an absolutely positioned item is positioned relative to its

ancestors that have a declared position property. Since none have that property now, it moves to the overall document position instead of the `<div class="herodiv">` container.

5. Also, note the footer using a **position: fixed** declaration.

## Submission

1. Commit and push your work to your wdd130 GitHub Pages enabled repository.

```
https://githubusername.github.io/wdd130/week03/overlay.html
```

2. Share and discuss your work with your peers on Microsoft Teams in the **Week 03 Forum** channel.

### Optional Resources

- [Normal Flow](#) – MDN
- [CSS Position](#) – MDN

Copyright © Brigham Young University-Idaho | All rights reserved